Introduction to Coding theory

Ji, Yonghyeon

April 30, 2026

We cover the following topics in this note.

- TBA.

Contents

1 Linear Codes

The fundamental problem of coding theory is to transmit information reliably across a noisy channel. A sender begins with a message, usually represented as a vector

$$\mathbf{x} = (x_1, \ldots, x_k) \in \mathbb{F}_2^k,$$

and applies an *encoder*

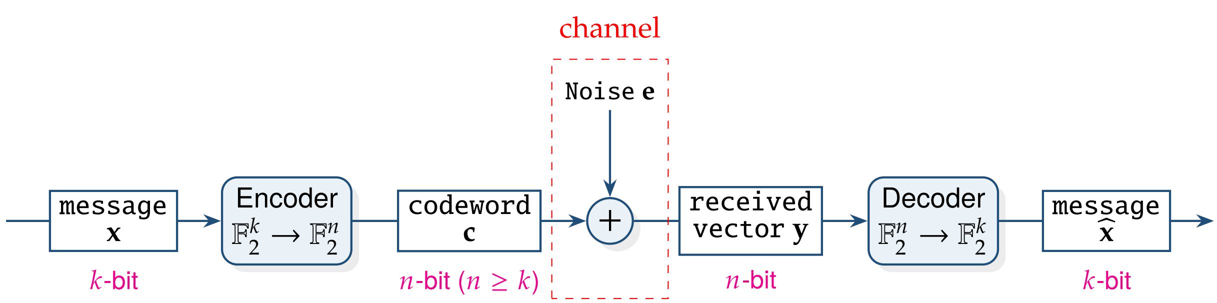
$$\text{Encode} : \mathbb{F}_2^k \longrightarrow \mathbb{F}_2^n, \qquad \mathbf{x} \longmapsto \mathbf{c},$$

where $n \geq k$. The vector $\mathbf{c}$ is called the *codeword*.

During transmission, the codeword passes through a noisy channel. The effect of the channel is modeled by adding an error vector $\mathbf{e} \in \mathbb{F}_2^n$ to the transmitted codeword, so that the receiver obtains

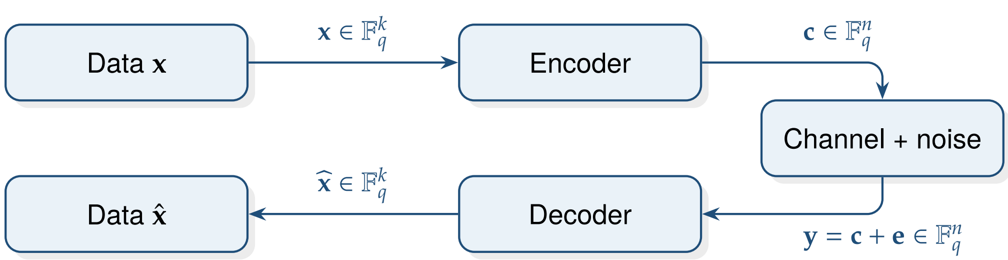
$$\mathbf{y} = \mathbf{c} \oplus \mathbf{e} \in \mathbb{F}_2^n.$$

The vector $\mathbf{y}$ is called the *received vector*. The purpose of the decoder is to recover the original message, or equivalently the transmitted codeword, from the corrupted vector $\mathbf{y}$.



A generic communication model has the form

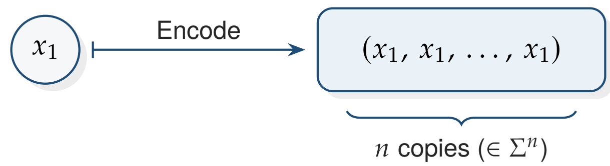
$$\mathbf{x} = (x_1, \ldots, x_k) \xrightarrow{\text{encoder}} \underbrace{\mathbf{c} = (c_1, \ldots, c_n)}_{\text{codeword } (n \geq k)} \xrightarrow{\text{noise } \mathbf{e}} \mathbf{y} = \mathbf{c} + \mathbf{e} \xrightarrow{\text{decoder}} \widehat{\mathbf{x}} = (\widehat{x}_1, \ldots, \widehat{x}_k).$$



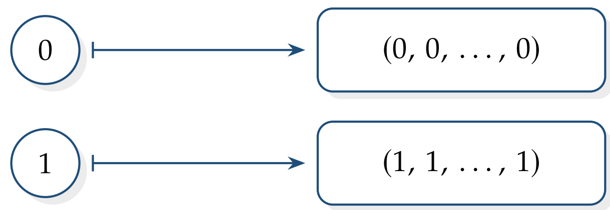
Example 1 (Repetition code). Let Σ be an alphabet. An *encoding* is a map

$$\begin{array}{rccc} \text{Encode} : & \Sigma^k & \longrightarrow & \Sigma^n \\ & (x_1, \ldots, x_k) & \longmapsto & (c_1, \ldots, c_n) \end{array}$$

with $n \geq k$.



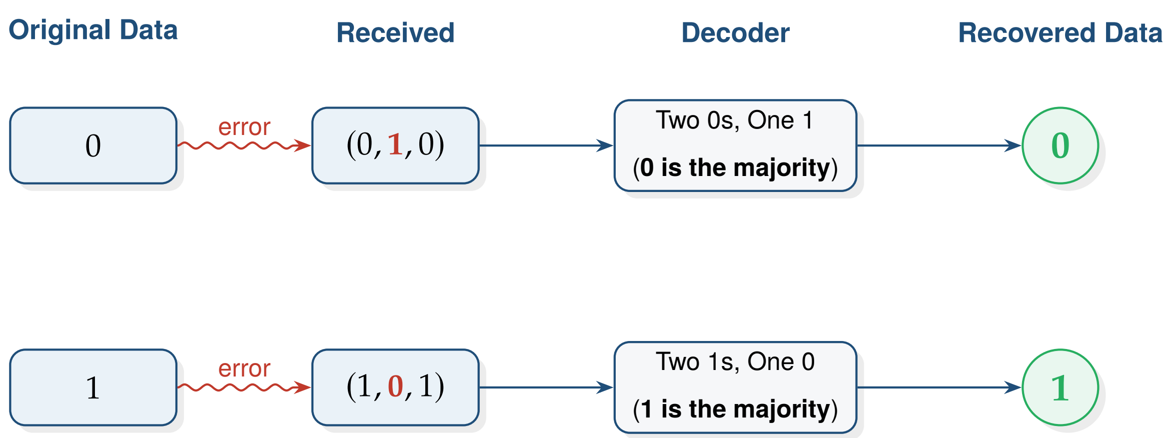
When $k = 1$, a very simple encoding repeats the same symbol



This is called the *repetition code*. If one symbol is changed during transmission, we can still recover the original message by *majority vote*. For instance,

$$(0, 0, 0) \rightsquigarrow (0, 1, 0), \qquad (1, 1, 1) \rightsquigarrow (1, 0, 1).$$

Since 0 appears most often in $(0, 1, 0)$, we decode it as 0, and since 1 appears most often in $(1, 0, 1)$, we decode it as 1.



1.1 Definition of Linear Codes

Linear Encoding

Definition 1. A **linear encoding** over $\mathbb{F}_q$ is a linear map

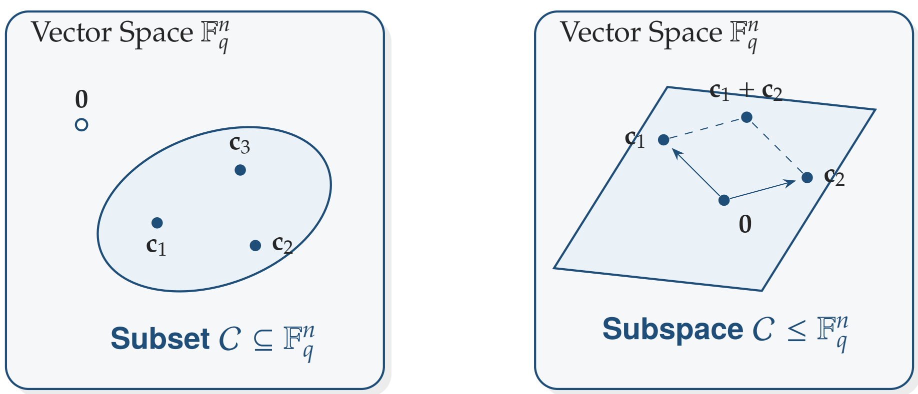
$$\mathbb{F}_q^k \longrightarrow \mathbb{F}_q^n, \qquad \mathbf{x} \longmapsto \mathbf{c}.$$

Linear Code

Definition 2. A **code of length** $n \in \mathbb{Z}_{>0}$ **over** $\mathbb{F}_q$ is a subset

$$C \subseteq \mathbb{F}_q^n.$$

The elements of C are called **codewords**. If $C \leq \mathbb{F}_q^n$, then C is called a **linear code**.



Remark 1. If $\dim_{\mathbb{F}_q}(C) = k$, then C is called a $[n,k]_q$**-code over** $\mathbb{F}_q$.

Proposition 1. *If C is an $[n,k]_q$-code, then*

$$|C| = q^k.$$

Proof. Choose a basis $\mathbf{b}_1, \ldots, \mathbf{b}_k$ of C. Every codeword can be written uniquely as

$$x_1\mathbf{b}_1 + \cdots + x_k\mathbf{b}_k, \qquad x_i \in \mathbb{F}_q.$$

Each x_i has q choices, and there are k independent coefficients. Therefore the number of codewords is q^k. $\square$

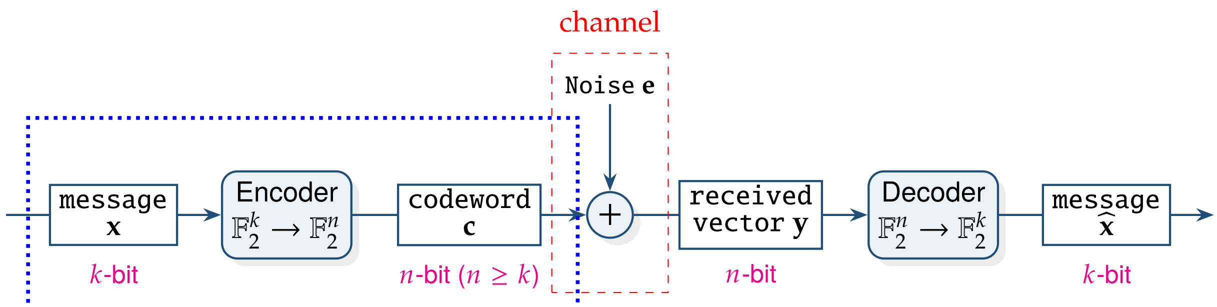
1.2 Generator Matrices

Note. In linear algebra, a choice of bases for finite-dimensional vector spaces V and W identifies each linear transformation $T : V \to W$ with a matrix.

Accordingly, in coding theory, after choosing bases for $\mathbb{F}_q^k$ and $\mathbb{F}_q^n$, a linear encoding

$$\text{Encode} : \mathbb{F}_q^k \to \mathbb{F}_q^n$$

is represented by a matrix $\mathbf{G} \in \mathbb{F}_q^{n\times k}$.

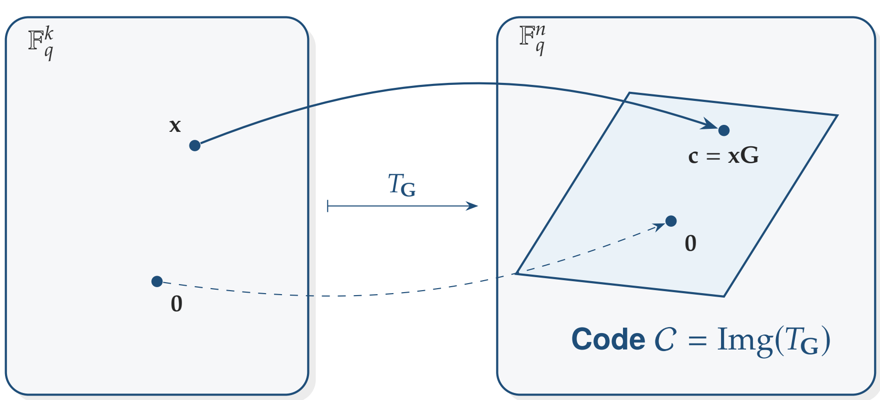


Generator Matrix

Definition 3. Let C be an $[n, k]_q$-code over $\mathbb{F}_q$. A **generator matrix** for C is a $k \times n$ matrix $\mathbf{G}$ whose rows form a basis of C.

Remark 2. If $\mathbf{x} = (x_1, \ldots, x_k) \in \mathbb{F}_q^k$, then encoding is given by

$$\mathbf{c} = \mathbf{x}\mathbf{G} \in \mathbb{F}_q^n \qquad \text{with } \mathbf{G} \in \mathbb{F}_q^{k\times n}.$$

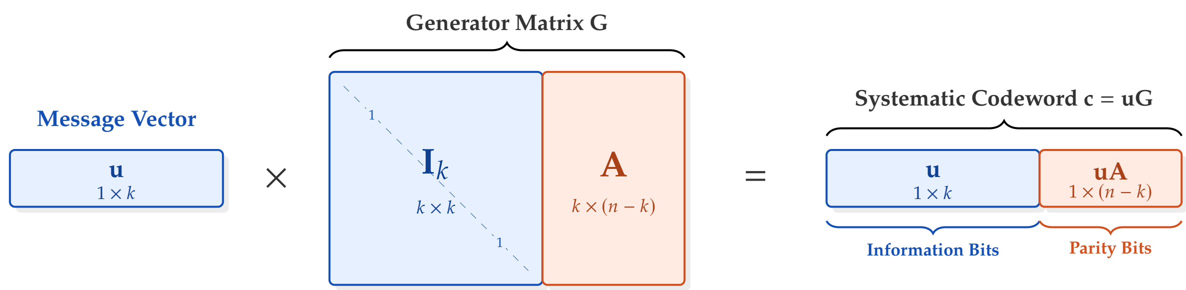


1.2.1 Systematic form

A particularly convenient generator matrix has the form

$$\mathbf{G} = [\mathbf{I}_k \mid \mathbf{A}],$$

where $\mathbf{I}_k$ is the $k \times k$ identity matrix and $\mathbf{A}$ is a $k \times (n-k)$ matrix.



Systematic Form

Definition 4. A code is said to be in **systematic form** if it has a generator matrix of the form

$$\mathbf{G} = [\mathbf{I}_k \mid \mathbf{A}]$$

Remark 3. In general, a linear code has many generator matrices, since different choices of a basis of the code give different matrices. Thus a generator matrix is not unique.

However, if the code is written in *systematic form* with the first k coordinate positions chosen as the information positions, then there is a unique generator matrix of the form $\mathbf{G} = [\mathbf{I}_k \mid \mathbf{A}]$.

Lemma 2. *Let $\mathbf{G}, \mathbf{G}' \in \mathbb{F}_q^{k\times n}$ be generator matrices of the same linear code $C \leq \mathbb{F}_q^n$. Then there exists a matrix*

$$\exists \mathbf{U} \in GL_k(\mathbb{F}_q) \quad \textit{such that} \quad \mathbf{G}' = \mathbf{U}\mathbf{G}.$$

Proof. Let

$$\mathbf{G} = \begin{pmatrix} — & \mathbf{g}_1 & — \\ — & \mathbf{g}_2 & — \\ & \vdots & \\ — & \mathbf{g}_k & — \end{pmatrix}, \qquad \mathbf{G}' = \begin{pmatrix} — & \mathbf{g}_1' & — \\ — & \mathbf{g}_2' & — \\ & \vdots & \\ — & \mathbf{g}_k' & — \end{pmatrix} \in \mathbb{F}_q^{k\times n}$$

be generator matrices of the same linear code $C \leq \mathbb{F}_q^n$. Then their rows are two bases of the same k-dimensional subspace $C \leq \mathbb{F}_q^n$, so each row of $\mathbf{G}'$ is a linear combination of the rows of $\mathbf{G}$. Thus, there exist scalars $u_{ij} \in \mathbb{F}_q$ such that

$$\begin{aligned}
\mathbf{g}_1' &= u_{11}\mathbf{g}_1 + u_{12}\mathbf{g}_2 + \cdots + u_{1k}\mathbf{g}_k,\\
\mathbf{g}_2' &= u_{21}\mathbf{g}_1 + u_{22}\mathbf{g}_2 + \cdots + u_{2k}\mathbf{g}_k,\\
&\vdots\\
\mathbf{g}_k' &= u_{k1}\mathbf{g}_1 + u_{k2}\mathbf{g}_2 + \cdots + u_{kk}\mathbf{g}_k.
\end{aligned}$$

Let $\mathbf{U} = (u_{ij}) =\in \mathbb{F}_q^{k\times k}$. Then

$$\mathbf{UG} = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1k}\\ u_{21} & u_{22} & \cdots & u_{2k}\\ \vdots & \vdots & \ddots & \vdots\\ u_{k1} & u_{k2} & \cdots & u_{kk}\end{pmatrix}\begin{pmatrix} — & \mathbf{g}_1 & —\\ — & \mathbf{g}_2 & —\\ & \vdots & \\ — & \mathbf{g}_k & —\end{pmatrix} = \begin{pmatrix} — & (u_{11}\mathbf{g}_1 + \cdots + u_{1k}\mathbf{g}_k) & —\\ — & (u_{21}\mathbf{g}_1 + \cdots + u_{1k}\mathbf{g}_k) & —\\ & \vdots & \\ — & (u_{k1}\mathbf{g}_1 + \cdots + u_{kk}\mathbf{g}_k) & —\end{pmatrix} = \begin{pmatrix} — & \mathbf{g}_1' & —\\ — & \mathbf{g}_2' & —\\ & \vdots & \\ — & \mathbf{g}_k' & —\end{pmatrix} = \mathbf{G}'.$$

So $\mathbf{G}' = \mathbf{UG}$. Similarly, since the rows of $\mathbf{G}$ are also linear combinations of the rows of $\mathbf{G}'$,

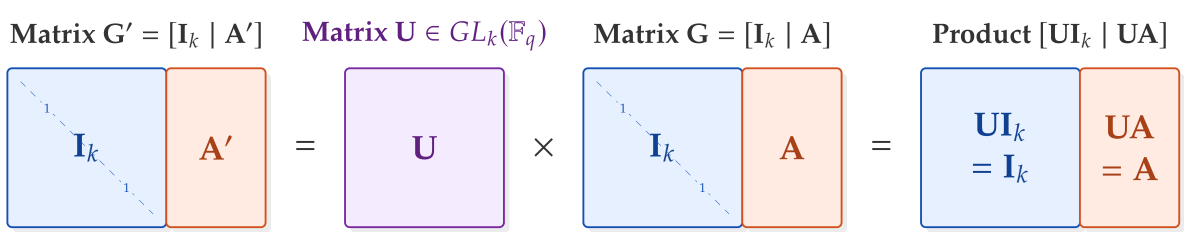
$$\exists \mathbf{V} \in \mathbb{F}_q^{k\times k} \quad \text{such that} \quad \mathbf{G} = \mathbf{VG}'.$$

Since $\mathbf{G}' = \mathbf{UG}$, we have $\mathbf{G} = \mathbf{VUG}$. Therefore $\mathbf{VU} = \mathbf{I}_k$. Similarly, the equation $\mathbf{G}' = \mathbf{UG} = \mathbf{UVG}$ gives $\mathbf{UV} = \mathbf{I}_k$. Therefore $\mathbf{U}$ is invertible, with inverse $\mathbf{V}$. Hence $\mathbf{U}, \mathbf{V} \in GL_k(\mathbb{F}_q)$. □

Note (Uniqueness of Generator Matrices in Systematic Form)**.** If $\mathbf{G} = [\mathbf{I}_k \mid \mathbf{A}]$ and $\mathbf{G}' = [\mathbf{I}_k \mid \mathbf{A}']$ are two generator matrices for the same code, then $\mathbf{G}' = \mathbf{UG}$ for some $\mathbf{U} \in GL_k(\mathbb{F}_q)$. Then

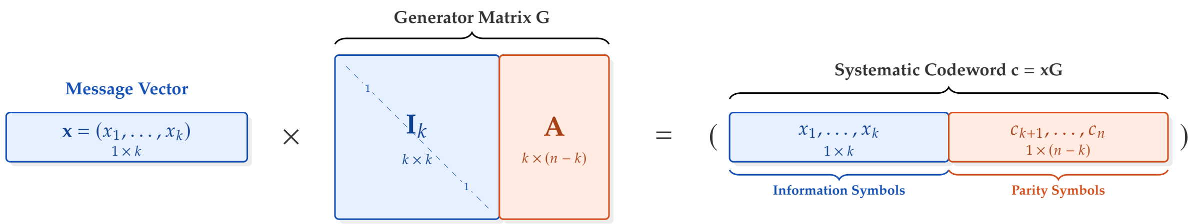
$$\mathbf{G}' = \mathbf{UG} \implies [\mathbf{I}_k \mid \mathbf{A}'] = \mathbf{U}[\mathbf{I}_k \mid \mathbf{A}] \implies [\mathbf{I}_k \mid \mathbf{A}'] = [\mathbf{UI}_k \mid \mathbf{UA}]$$

so $\mathbf{U} = \mathbf{I}_k$. Hence $\mathbf{G}' = \mathbf{G}$, and therefore $\mathbf{A}' = \mathbf{A}$.



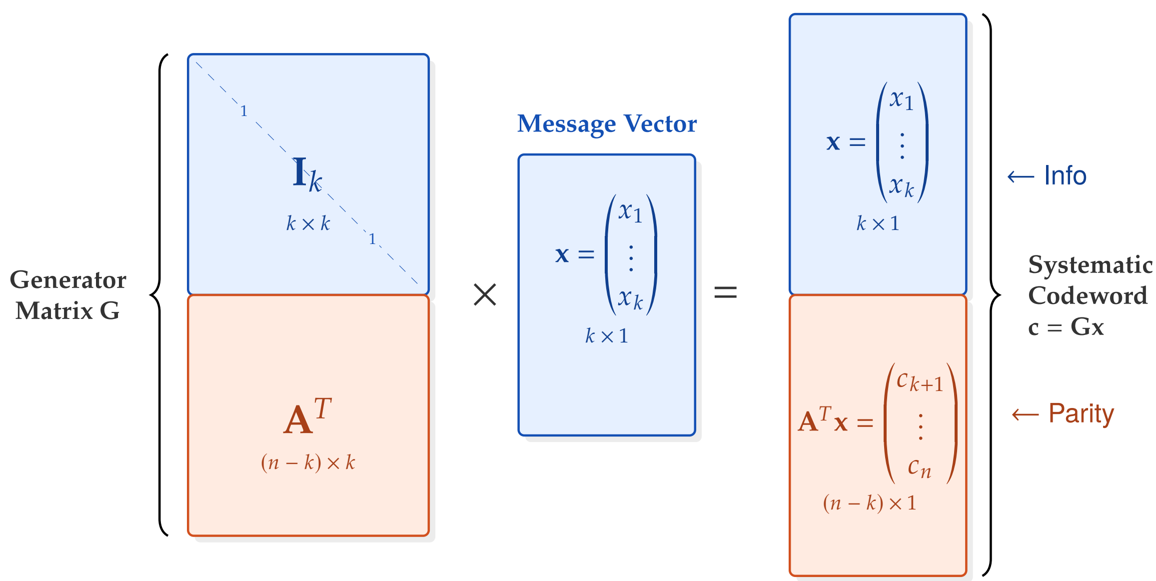
Remark 4. Let the first k coordinates of the codeword are exactly the information symbols. If $\mathbf{x} = (x_1, \dots, x_k)$, then

$$\mathbf{x}G = (x_1, \dots, x_k, c_{k+1}, \dots, c_n).$$



So the information symbols appear in the transmitted codeword, followed by the parity symbols.

Note. Let $\mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_k \end{pmatrix} \in \mathbb{F}_q^k$ be the message vector, written as a column vector. With the column-vector convention, the encoding map is represented by a generator matrix $\mathbf{G}$ such that $\mathbf{G}\mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_k \\ c_{k+1} \\ \vdots \\ c_n \end{pmatrix}$.



1.2.2 Examples: Repetition Code and Single Parity-check Code

Example 2 (Repetition code). The $[n,1]_q$ repetition code is the linear code defined by

$$\text{Encode} : \mathbb{F}_q \longrightarrow \mathbb{F}_q^n, \qquad (x_1) \longmapsto (x_1, x_1, \dots, x_1).$$

Its generator matrix is

$$\mathbf{G} = [1\ 1\ \cdots\ 1]_{1\times n}.$$

Indeed,

$$(x_1)\mathbf{G} = (x_1)[1\ 1\ \cdots\ 1] = (x_1, x_1, \dots, x_1).$$

Hence the code is

$$C = \{(a, a, \dots, a) \in \mathbb{F}_q^n : a \in \mathbb{F}_q\} \le \mathbb{F}_q^n.$$

Example 3 (Single Parity-check Code). The single parity-check code $[n, n-1]_q$ is the linear code

$$C = \left\{(x_1, \dots, x_{n-1}, \textstyle\sum_{i=1}^{n-1} x_i) \in \mathbb{F}_q^n : (x_1, \dots, x_k) \in \mathbb{F}_q^k\right\} \le \mathbb{F}_q^n.$$

Equivalently, it is defined by the encoding map

$$\text{Encode} : \mathbb{F}_q^{n-1} \longrightarrow \mathbb{F}_q^n, \qquad (x_1, \dots, x_{n-1}) \longmapsto (x_1, \dots, x_k, \textstyle\sum_{i=1}^{n-1} x_i).$$

A generator matrix in systematic form is

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & \cdots & 0 & 1 \\ 0 & 1 & \cdots & 0 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 1 \end{bmatrix}_{k\times n} = \begin{bmatrix} & 1 \\ \mathbf{I}_k & \vdots \\ & 1 \end{bmatrix} = [\mathbf{I}_k \mid \mathbf{1}],$$

where $\mathbf{1} \in \mathbb{F}_q^k$ denotes the column vector all of whose entries are 1. Indeed, for $\mathbf{x} = (x_1, \dots, x_k) \in \mathbb{F}_q^k$,

$$\mathbf{x}\mathbf{G} = (x_1, \dots, x_k) \begin{bmatrix} & 1 \\ \mathbf{I}_k & \vdots \\ & 1 \end{bmatrix} = (x_1, \dots, x_k, x_1 + \cdots + x_k).$$

Hence the first $n-1$ coordinates are the information symbols, and the last coordinate is their parity symbol.

1.3 Parity-check Matrices

A linear code can be described either by its generators or by the linear equations that all codewords satisfy.

Parity-check Matrix

Definition 5. Let C be an $[n,k]_q$-code. A matrix $H \in \mathbb{F}_q^{(n-k)\times n}$ is called a **parity-check matrix** for C if

$$C = \{\mathbf{v} \in \mathbb{F}_q^n : H\mathbf{v}^\mathsf{T} = 0\}.$$

Thus codewords are exactly the vectors satisfying a system of $n-k$ homogeneous linear equations.

1.3.1 Parity-check Matrix from a Systematic Generator Matrix

Note. Suppose

$$\mathbf{G} = [\mathbf{I}_k \mid \mathbf{A}],$$

is a generator matrix in systematic form, where $\mathbf{A} \in \mathbb{F}_q^{k\times(n-k)}$. Then define

$$\mathbf{H} = [-\mathbf{A}^\mathsf{T} \mid \mathbf{I}_{n-k}].$$

Parity-check Matrix of a Systematic Generator Matrix

Proposition 3. *Let C be an $[n,k]_q$-code, and suppose that C is generated by the systematic generator matrix*

$$\mathbf{G} = [\mathbf{I}_k \mid \mathbf{A}] \in \mathbb{F}_q^{k\times n},$$

where $\mathbf{A} \in \mathbb{F}_q^{k\times(n-k)}$. Then

$$\mathbf{H} = [-\mathbf{A}^\mathsf{T} \mid \mathbf{I}_{n-k}] \in \mathbb{F}_q^{(n-k)\times n}$$

is a parity-check matrix of C.

Proposition 4 (Parity-check matrix of a systematic generator matrix)**.** *Let C be an $[n,k]_q$-code, and suppose that C is generated by the systematic generator matrix*

$$G = [I_k \mid A] \in M_{k\times n}(\mathbb{F}_q),$$

where $A \in M_{k\times(n-k)}(\mathbb{F}_q)$. Then

$$H = [-A^\mathsf{T} \mid I_{n-k}] \in M_{(n-k)\times n}(\mathbb{F}_q)$$

is a parity-check matrix of C.

Proof. By definition, it suffices to show that

$$GH^{\intercal} = 0.$$

Now

$$H^{\intercal} = \begin{bmatrix} -A \\ I_{n-k} \end{bmatrix},$$

so

$$GH^{\intercal} = [I_k \mid A] \begin{bmatrix} -A \\ I_{n-k} \end{bmatrix} = I_k(-A) + AI_{n-k} = -A + A = 0.$$

□

Note (Linear Code as Short Exact Sequence)**.** A linear $[n,k]_q$ code is most naturally described by a short exact sequence

$$0 \longrightarrow \mathbb{F}_q^k \xrightarrow{\mathsf{Encode}} \mathbb{F}_q^n \xrightarrow{H} \mathbb{F}_q^{n-k} \longrightarrow 0,$$

A message $\mathbf{x} \in \mathbb{F}_q^k$ is encoded as

$$\mathbf{c} = \mathsf{Enc}(\mathbf{x}) \in C.$$

After transmission through the channel, one receives

$$\mathbf{y} = \mathbf{c} + \mathbf{e} \in \mathbb{F}_q^n,$$

where $\mathbf{e} \in \mathbb{F}_q^n$ is the error vector. Applying the parity-check map yields

$$H(\mathbf{y}) = H(\mathbf{c} + \mathbf{e}) = H(\mathbf{e}),$$

since $H(\mathbf{c}) = 0$ for every $\mathbf{c} \in C$.
Thus decoding may be viewed as follows: from the syndrome $H(\mathbf{y})$, choose an error estimate $\widehat{\mathbf{e}}$ with the same syndrome, typically of smallest Hamming weight, and define

$$\widehat{\mathbf{c}} = \mathbf{y} - \widehat{\mathbf{e}} \in C.$$

Then recover the message by

$$\widehat{\mathbf{x}} = \mathsf{Enc}^{-1}(\widehat{\mathbf{c}}).$$

Hence error correction is the procedure of moving the received word from its ambient coset in $\mathbb{F}_q^n/C$ back to the code $C = \ker(H)$, and then inverting the encoding map.

Cardinality of the matrix space $\mathrm{Mat}_m(\mathbb{F}_q)$

Proposition 5. *Let $m, n \in \mathbb{Z}_{>0}$, and let q be a prime power. The number of $m \times n$ matrices over the finite field $\mathbb{F}_q$ is*

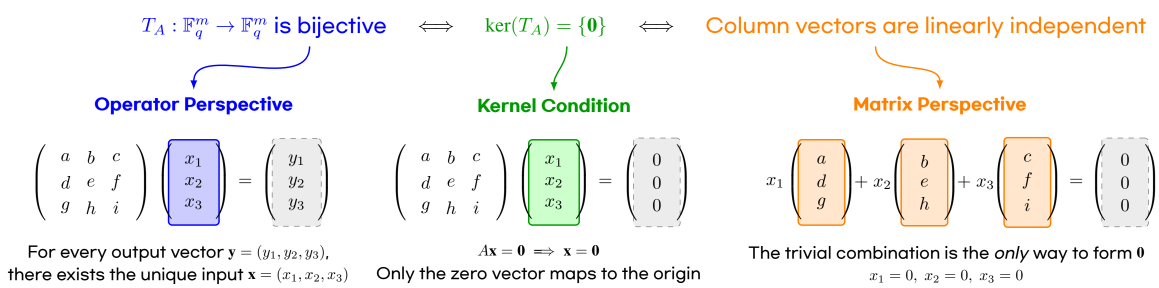
$$\left|\mathrm{Mat}_{m\times n}(\mathbb{F}_q)\right| = q^{mn}.$$

In particular, $\left|\mathrm{Mat}_m(\mathbb{F}_q)\right| = q^{m^2}$.

Remark 5. Let

$$A = \begin{pmatrix} | & | & & | \\ \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_m \\ | & | & & | \end{pmatrix} \in \mathrm{Mat}_{m\times m}(\mathbb{F}_q),$$

where $\mathbf{v}_i \in \mathbb{F}_q^m$ are the columns of A. Then A is invertible if and only if its columns are linearly independent.



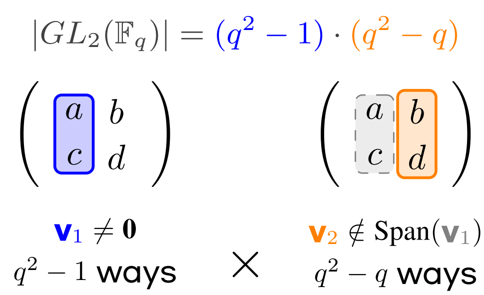
Observation (Computation of $|GL_2(\mathbb{F}_q)|$).

A 2×2 matrix has the form

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \qquad a, b, c, d \in \mathbb{F}_q.$$

Write the columns as

$$\mathbf{v}_1 = \begin{pmatrix} a \\ c \end{pmatrix}, \qquad \mathbf{v}_2 = \begin{pmatrix} b \\ d \end{pmatrix}.$$



(i) The vector $\mathbf{v}_1$ can be any nonzero vector in $\mathbb{F}_q^2$. Hence there are $q^2 - 1$ choices.

(ii) Since $\mathrm{span}(\mathbf{v}_1) = \{\lambda \mathbf{v}_1 : \lambda \in \mathbb{F}_q\}$ has exactly q elements, there are $q^2 - q$ choices for $\mathbf{v}_2$.

Multiplying these choices, we obtain

$$|GL_2(\mathbb{F}_q)| = (q^2 - 1)(q^2 - q).$$

Observation (Computation of $|GL_3(\mathbb{F}_q)|$).

$$|GL_3(\mathbb{F}_q)| = (q^3-1)\cdot(q^3-q)\cdot(q^3-q^2)$$

$$\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \qquad \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \qquad \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$$

$$\underset{q^3-1 \text{ ways}}{\mathbf{v}_1 \neq \mathbf{0}} \times \underset{q^3-q \text{ ways}}{\mathbf{v}_2 \notin \operatorname{Span}(\mathbf{v}_1)} \times \underset{q^3-q^2 \text{ ways}}{\mathbf{v}_3 \notin \operatorname{Span}(\mathbf{v}_1,\mathbf{v}_2)}$$

Observation (Computation of $|GL_4(\mathbb{F}_q)|$).

$$|GL_4(\mathbb{F}_q)| = (q^4-1)\cdot(q^4-q)\cdot(q^4-q^2)\cdot(q^4-q^3)$$

$$\begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix} \quad \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix} \quad \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix} \quad \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix}$$

$$\underset{q^4-1 \text{ ways}}{\mathbf{v}_1 \neq \mathbf{0}} \times \underset{q^4-q \text{ ways}}{\mathbf{v}_2 \notin \operatorname{Span}(\mathbf{v}_1)} \times \underset{q^4-q^2 \text{ ways}}{\mathbf{v}_3 \notin \operatorname{Span}(\mathbf{v}_1,\mathbf{v}_2)} \times \underset{q^4-q^3 \text{ ways}}{\mathbf{v}_4 \notin \operatorname{Span}(\mathbf{v}_i)_{i=1}^3}$$

Counting invertible matrices over a finite field $\mathbb{F}_q$

Theorem 6. *Let $m, n \in \mathbb{Z}_{>0}$, and let q be a prime power. The number of invertible $m \times m$ matrices over $\mathbb{F}_q$ is*

$$|\mathrm{GL}_m(\mathbb{F}_q)| = (q^m-1)(q^m-q)(q^m-q^2)\cdots(q^m-q^{m-1}).$$

Equivalently, $|\mathrm{GL}_m(\mathbb{F}_q)| = \displaystyle\prod_{i=0}^{m-1}(q^m-q^i)$.

Remark 6. Recall that

$$|\mathrm{Mat}_m(\mathbb{F}_q)| = q^{m^2}, \quad \text{and} \quad |GL_m(\mathbb{F}_q)| = \prod_{i=0}^{m-1}(q^m-q^i),$$

and hence

$$\begin{aligned}
\frac{|GL_m(\mathbb{F}_q)|}{|\operatorname{Mat}_m(\mathbb{F}_q)|} = \frac{\prod_{i=0}^{m-1}(q^m - q^i)}{q^{m^2}} &= \frac{1}{q^{m^2}}\left[(q^m-1)(q^m-q)(q^m-q^2)\cdots(q^m-q^{m-1})\right] \\
&= \frac{1}{q^{m^2}}\left[q^m\left(1-\frac{1}{q^m}\right)\left(1-\frac{1}{q^{m-1}}\right)\left(1-\frac{1}{q^{m-2}}\right)\cdots\left(1-\frac{1}{q^1}\right)\right] \\
&= \left(1-\frac{1}{q^m}\right)\left(1-\frac{1}{q^{m-1}}\right)\left(1-\frac{1}{q^{m-2}}\right)\cdots\left(1-\frac{1}{q^1}\right) \\
&= \prod_{j=1}^{m}(1-q^{-j}).
\end{aligned}$$

m	$\lvert\operatorname{Mat}_m(\mathbb{F}_q)\rvert$	$\lvert GL_m(\mathbb{F}_q)\rvert$	$\frac{\lvert GL_m(\mathbb{F}_q)\rvert}{\lvert\operatorname{Mat}_m(\mathbb{F}_q)\rvert}$
1	q	$q-1$	$1-q^{-1}$
2	q^4	$(q^2-1)(q^2-q)$	$(1-q^{-1})(1-q^{-2})$
3	q^9	$(q^3-1)(q^3-q)(q^3-q^2)$	$(1-q^{-1})(1-q^{-2})(1-q^{-3})$
4	q^{16}	$(q^4-1)(q^4-q)(q^4-q^2)(q^4-q^3)$	$(1-q^{-1})(1-q^{-2})(1-q^{-3})(1-q^{-4})$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
m	q^{m^2}	$\prod_{i=0}^{m-1}(q^m-q^i)$	$\prod_{j=1}^{m}(1-q^{-j})$

Example 4 (Invertible binary matrices)**.** For $q = 2$, we have

$$|\operatorname{Mat}_m(\mathbb{F}_2)| = 2^{m^2}, \qquad \frac{|GL_m(\mathbb{F}_2)|}{|\operatorname{Mat}_m(\mathbb{F}_2)|} = \prod_{j=1}^{m}(1-2^{-j}).$$

m	$\lvert\operatorname{Mat}_m(\mathbb{F}_2)\rvert$	$\lvert GL_m(\mathbb{F}_2)\rvert$	$\frac{\lvert GL_m(\mathbb{F}_2)\rvert}{\lvert\operatorname{Mat}_m(\mathbb{F}_2)\rvert}$
1	2	1	0.500000
2	16	6	0.375000
3	512	168	0.328125
4	65,536	20,160	0.307617
$\vdots$	$\vdots$	$\vdots$	$\vdots$
20	–	–	0.288788

```
q = 2

print(f"{'m':>2} {'|Mat_m(F_q)|':>15} {'|GL_m(F_q)|':>15} {'ratio':>12}")
print("-" * 48)

for m in range(1, 7):
    Mat = q^(m^2)
    GL = prod(q^m - q^i for i in range(m))
    ratio = GL / Mat
    print(f"{m:>2} {Mat:>15} {GL:>15} {ratio.n():>12.6f}")
```

```
m     |Mat_m(F_q)|      |GL_m(F_q)|         ratio
------------------------------------------------
1                2                1      0.500000
2               16                6      0.375000
3              512              168      0.328125
4            65536            20160      0.307617
5         33554432          9999360      0.298004
6      68719476736      20158709760      0.293348
```

Proposition 7. *Let $q \geq 2$ be fixed, and define*

$$a_m := \frac{|GL_m(\mathbb{F}_q)|}{|\operatorname{Mat}_m(\mathbb{F}_q)|} = \prod_{j=1}^{m}(1 - q^{-j}).$$

Then the sequence $\langle a_m \rangle_{m \geq 1}$ converges as $m \longrightarrow \infty$. In other words, there exists $\lim\limits_{n\to\infty} \prod_{j=1}^{m}(1-q^{-j}) \in \mathbb{R}$.

Proof. For every $j \geq 1$,

$$q \geq 2 \implies q^j \geq 2^j > 1 \implies 0 < \frac{1}{q^j} < 1.$$

Therefore

$$0 < \frac{1}{q^j} < 1 \implies -1 < -\frac{1}{q^j} < 0 \implies 0 < 1 - \frac{1}{q^j} < 1.$$

It follows that for each $m \geq 1$,

$$a_m = \prod_{j=1}^{m}(1 - q^{-j}) > 0.$$

So the sequence $\langle a_m \rangle$ is bounded below by 0. Moreover,

$$a_{m+1} = \prod_{j=1}^{m+1}(1-q^{-j}) = \left[\prod_{j=1}^{m}(1-q^{-j})\right]\cdot(1-q^{-(m+1)}) = a_m(1-q^{-(m+1)}).$$

Since $0 < 1-q^{-(m+1)} < 1$, we obtain $a_{m+1} < a_m$. Thus $\langle a_m \rangle$ is decreasing. Therefore $\langle a_m \rangle$ is a monotone decreasing sequence which is bounded below. Hence it converges. □

Example 5. For small prime q, the limit is approximately

$$q = 2: \qquad \prod_{j=1}^{\infty}(1-2^{-j}) \approx 0.288788095$$

$$q = 3: \qquad \prod_{j=1}^{\infty}(1-3^{-j}) \approx 0.560126077$$

$$q = 5: \qquad \prod_{j=1}^{\infty}(1-5^{-j}) \approx 0.760332795$$

```
q = 2
r = 3
s = 5
R = RealField(100)

L = prod(R(1) - R(q)^(-j) for j in range(1, 100))
M = prod(R(1) - R(r)^(-j) for j in range(1, 100))
N = prod(R(1) - R(s)^(-j) for j in range(1, 100))

print(L)
print(M)
print(N)
```

```
0.28878809508660242127889972193
0.56012607792794894496979224331
0.76033279587123242010148829629
```

1.4 Weights and Distances

2 Cyclic Codes

2.1 Historical Development of Cyclic Codes

Year	Milestone	Description
1948	Birth of Coding Theory	Shannon established information theory and proved the existence of reliable codes [1].
1950	Hamming Codes	Hamming introduced the first systematic single-error-correcting linear codes [2].
1954	Reed–Muller Codes	Reed and Muller introduced multiple-error-correcting codes from Boolean functions [3][4].
1957	Algebraic Foundation of Cyclic Codes	Prange identified cyclic codes with ideals in $\mathbb{F}_q[X]/\langle X^n - 1\rangle$ [5].
1959–1960	BCH Codes	Hocquenghem, Bose, and Ray-Chaudhuri developed BCH codes with designed distance [6][7].
1960	Reed–Solomon Codes	Reed and Solomon introduced powerful nonbinary cyclic codes [8].
1961	Peterson's Decoding Algorithm	Peterson developed algebraic BCH decoding and published a foundational textbook [9].
1968	Berlekamp–Massey Algorithm	Berlekamp and Massey introduced an efficient error-locator algorithm [10][11].
1970s	Industrial Standardization of CRC	CRC became standard because cyclic codes allow efficient LFSR implementation.
1977	Goppa Codes and the McEliece Cryptosystem	Goppa introduced algebraic geometry codes, and McEliece applied them to cryptography [12][13].
Since 1990s	Modern Developments	Cyclic codes remained important despite the rise of turbo and LDPC codes [14][15].

2.2 What Is a Cyclic Code?

Note. A cyclic code is a special subclass of linear codes that satisfies two key properties

(i) **linearity** and

(ii) **invariance under cyclic shift**.

Cyclic code

Definition 6. A linear code C of length n is called a cyclic code if it satisfies both of the following

(i) **Linearity**: for any two codewords $\mathbf{c}_1, \mathbf{c}_2 \in C$, we have $\mathbf{c}_1 + \mathbf{c}_2 \in C$.

(ii) **Cyclic shift property**: if $\mathbf{c} = (c_0, c_1, \ldots, c_{n-1}) \in C$, then the vector

$$\mathbf{c}^{\ggg 1} = (c_{n-1}, c_0, c_1, \ldots, c_{n-2})$$

must also belong to C.

Example 6 (Binary $[7,4]_2$ cyclic code)**.** Suppose

$$\mathbf{c} = (1,0,0,1,0,1,1)$$

is a codeword. Then

- $\mathbf{c}^{\ggg 1} = (1,1,0,0,1,0,1)$
- $\mathbf{c}^{\ggg 2} = (1,1,1,0,0,1,0)$
- $\mathbf{c}^{\ggg 3} = (0,1,1,1,0,0,1)$

All of these vectors must belong to the code space C. Let $n = 4$ and consider the set

$$C = \{(0,0,0,0),\ (1,0,1,0),\ (0,1,0,1),\ (1,1,1,1)\}.$$

- Linearity check: $(1,0,1,0) \oplus (0,1,0,1) = (1,1,1,1) \in C$.
- Cyclic-shift check:
  - $(1,0,1,0)$ shifted once gives $(0,1,0,1) \in C$.
  - $(0,1,0,1)$ shifted once gives $(1,0,1,0) \in C$.
  - $(1,1,1,1)$ shifted once gives $(1,1,1,1) \in C$.

Hence this is a cyclic code.

2.2.1 Example 1.3: A Linear Code That Is Not Cyclic

Let

$$C' = \{(0,0,0,0),\ (1,1,0,0),\ (0,0,1,1),\ (1,1,1,1)\}.$$

Shifting $(1,1,0,0)$ one step cyclically gives $(0,1,1,0)$, which is not in C'. Therefore C' is linear, but **not** cyclic.

2.3 Implementation Advantage of Cyclic-Shift Invariance

Cyclic codes were originally studied not because of abstract algebra, but because of **hardware efficiency**. Prange's key observation in the 1950s was that if a code is closed under cyclic shift, then encoding and error detection can be implemented using nothing more than shift registers.

The Drawback of General Linear Codes

For a general linear $[n, k]_q$ code, encoding is done by matrix multiplication:

$$\mathbf{c} = \mathbf{m} \cdot G.$$

This means:

- the $k \times n$ generator matrix G must be stored in memory,
- $k \cdot n$ additions and multiplications are needed,
- computation cannot begin until the whole message block has arrived.

For example, for a $[31, 21]_2$ code, one must store $21 \times 31 = 651$ matrix entries and perform 651 binary operations for every block.

The Cyclic-Code Alternative: Shift Registers

For cyclic codes, every codeword is a multiple of the generator polynomial $g(X)$, and encoding reduces to polynomial multiplication or polynomial division with remainder. These operations can be implemented using an LFSR (linear feedback shift register):

- only $r = n - k$ one-bit registers are required,
- the input stream is processed one bit per clock,
- after the last bit passes through the circuit, the register contents are exactly the parity bits or syndrome.

For a $[31, 21]_2$ code, 10 registers plus a few XOR gates are enough.

Example 7 (Matrix multiplication vs. LFSR for a $[7, 4]_2$ code)**.**

Item	General linear code	Cyclic code (LFSR)
Storage	$G \in \mathbb{F}_2^{4\times 7} \to 28$ bits	$g(X) = 1 + X + X^3 \to 4$ bits
Operations per block	$4 \times 7 = 28$ AND/XOR	7 clocks (one XOR per bit)
Processing mode	Batch (full block first)	Real-time, bit-by-bit
Latency	Proportional to block length	Almost zero
Circuit complexity	n XOR trees, k inputs	r flip-flops plus XOR gates

Remark 7. The key correspondence is

$$\text{cyclic shift} \longleftrightarrow X \cdot c(X) \pmod{X^n - 1}.$$

The basic action of a shift register—moving all bits by one position and injecting feedback—is exactly multiplication by X followed by reduction modulo $X^n - 1$. This is why cyclic codes admit LFSR implementations, whereas general linear codes do not. This is the fundamental reason cyclic codes became so important, and why CRC remains the standard error-detection mechanism in protocols such as Ethernet and USB.

2.4 Polynomial Representation of Cyclic Codes

A vector of length n,

$$\mathbf{c} = (c_0, c_1, \dots, c_{n-1}),$$

can be mapped to a polynomial of degree at most $n-1$:

$$c(X) = c_0 + c_1X + c_2X^2 + \cdots + c_{n-1}X^{n-1}.$$

Under this representation, a one-step cyclic right shift corresponds exactly to

$$X \cdot c(X) \pmod{X^n - 1}.$$

Therefore all algebraic operations on a cyclic code naturally live in the quotient ring

$$R_n = \mathbb{F}_q[X]/\langle X^n - 1\rangle.$$

Example 8 (Vector $\leftrightarrow$ Polynomial conversion, $n = 7$)**.**

Vector	Polynomial
$(1,0,1,1,0,0,0)$	$1 + X^2 + X^3$
$(0,1,0,0,1,1,0)$	$X + X^4 + X^5$
$(1,1,1,1,1,1,1)$	$1 + X + X^2 + X^3 + X^4 + X^5 + X^6$

Example 9 (Cyclic shift as a polynomial operation)**.** Let $n = 7$ and $c(X) = 1 + X^2 + X^3$. Then $Xc(X) = X + X^3 + X^4$. Since the degree is still below 7, reduction modulo $X^7 - 1$ changes nothing. The corresponding vector is $(0,1,0,1,1,0,0)$, which is exactly the one-step cyclic shift of $(1,0,1,1,0,0,0)$.

Example 10 (When the highest-degree term wraps around)**.** Let $n = 7$ and $c(X) = X^4 + X^5 + X^6$. Then $Xc(X) = X^5 + X^6 + X^7$. Since $X^7 \equiv 1 \pmod{X^7 - 1}$,

$$Xc(X) \bmod (X^7 - 1) = 1 + X^5 + X^6.$$

Thus

$$(0,0,0,0,1,1,1) \xrightarrow{\text{1-step shift}} (1,0,0,0,0,1,1).$$

2.5 Generator Polynomials

Generator Polynomial

Definition 7. Every cyclic code is completely determined by a unique monic polynomial $g(X)$ of **smallest degree** in the code. This polynomial is called the **generator polynomial** of the code.

Key properties:

- $g(X)$ must divide $X^n - 1$:
$$X^n - 1 = g(X)h(X).$$
- If $\deg g = r = n - k$, then the code dimension is k.
- Every codeword is a multiple of $g(X)$:
$$c(X) = m(X)g(X).$$

For **non-systematic encoding**, one simply multiplies the message polynomial by the generator polynomial: $c(X) = m(X)g(X)$.

Example 11 (Factorization of $X^7 - 1$ and possible generator polynomials). Over $\mathbb{F}_2$, we have $-1 = 1$, so $X^7 - 1 = X^7 + 1$, and it factors as

$$X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3).$$

Any divisor of this polynomial can serve as a generator polynomial:

$g(X)$	$\deg g$	$[n,k]_2$	2^k
$1 + X$	1	$[7,6]_2$	64
$1 + X + X^3$	3	$[7,4]_2$	16
$1 + X^2 + X^3$	3	$[7,4]_2$	16
$(1+X)(1+X+X^3) = 1+X^2+X^3+X^4$	4	$[7,3]_2$	8
$(1+X)(1+X^2+X^3) = 1+X+X^2+X^4$	4	$[7,3]_2$	8
$(1 + X + X^3)(1 + X^2 + X^3) = 1 + X + \cdots + X^6$	6	$[7,1]_2$	2
$X^7 + 1$	7	$[7,0]_2$	1 (trivial)

Example 12 (Non-systematic encoding for a $[7,4]_2$ code). Let $g(X) = 1+X+X^3$ and $\mathbf{m} = (1,1,0,1)$,

$m(X) = 1 + X + X^3$. Then

$$\begin{aligned}
c(X) = m(X)g(X) &= (1 + X + X^3)(1 + X + X^3) \\
&= 1 + X + X^3 + X + X^2 + X^4 + X^3 + X^4 + X^6 \\
&= 1 + X^2 + X^6.
\end{aligned}$$

Thus the codeword vector is $\mathbf{c} = (1, 0, 1, 0, 0, 0, 1)$.

2.6 Parity-Check Polynomials

Parity-Check Polynomial

Definition 8. The quotient obtained by dividing $X^n - 1$ by the generator polynomial is called the **parity-check polynomial**:

$$h(X) = \frac{X^n - 1}{g(X)}.$$

Its degree is k. A polynomial $v(X)$ is a valid codeword if and only if $v(X)h(X) \equiv 0 \pmod{X^n - 1}$.

Example 13 (Parity-check polynomial for a $[7,4]_2$ code)**.** If $g(X) = 1 + X + X^3$, then

$$h(X) = \frac{X^7 + 1}{1 + X + X^3} = 1 + X + X^2 + X^4.$$

Verification: $g(X)h(X) = (1 + X + X^3)(1 + X + X^2 + X^4) = X^7 + 1$. Indeed, term-by-term cancellation in $\mathbb{F}_2$ leaves only $1 + X^7$.

Example 14 (Validating a codeword)**.** Consider $c(X) = X + X^2 + X^4$. Then

$$c(X)h(X) = (X + X^2 + X^4)(1 + X + X^2 + X^4).$$

Expanding and reducing modulo $X^7 + 1$ yields 0, so $c(X)$ is a valid codeword.

2.7 Generator Matrices for Cyclic Codes

Let $g(X) = g_0 + g_1X + \cdots + g_{n-k}X^{n-k}$. The generator matrix G is built by taking the coefficient vector of $g(X)$ and shifting it one position to the right in each row:

$$G = \begin{bmatrix} g_0 & g_1 & g_2 & \cdots & g_{n-k} & 0 & \cdots & 0 \\ 0 & g_0 & g_1 & \cdots & g_{n-k-1} & g_{n-k} & \cdots & 0 \\ \vdots & & \ddots & & & & \ddots & \vdots \\ 0 & \cdots & 0 & g_0 & g_1 & \cdots & \cdots & g_{n-k} \end{bmatrix}.$$

The i-th row corresponds to the coefficient vector of $X^i g(X)$.

Remark 8. Why is this a valid generator matrix?

(1) $g(X)$ is a codeword, and by cyclic-shift invariance so are $Xg(X), X^2g(X), \ldots, X^{k-1}g(X)$.

(2) These vectors are linearly independent because their highest nonzero terms occur in different positions.

(3) A k-dimensional code containing k linearly independent codewords has found a basis.

Example 15 (Generator matrix for a $[7,4]_2$ cyclic code)**.** Let $g(X) = 1 + X + X^3$, whose coefficient vector is $(1,1,0,1)$. Then

$$G = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}.$$

Rows correspond to $g(X), Xg(X), X^2g(X), X^3g(X)$ respectively.

Example 16 (Encoding with the generator matrix)**.** Let $\mathbf{m} = (1,0,1,1)$. Then

$$\mathbf{c} = \mathbf{m}G = (1,0,1,1)\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} = (1,1,1,1,1,1,1).$$

Thus $c(X) = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6$, which is divisible by $g(X)$.

2.8 Parity-Check Matrices for Cyclic Codes

Let $h(X) = h_0 + h_1X + \cdots + h_kX^k$. To construct the parity-check matrix H, reverse the coefficient order of $h(X)$ and shift that reversed vector across the rows:

$$H = \begin{bmatrix} h_k & h_{k-1} & \dots & h_0 & 0 & \dots & 0 \\ 0 & h_k & h_{k-1} & \dots & h_0 & \dots & 0 \\ \vdots & & \ddots & & & \ddots & \vdots \\ 0 & \dots & 0 & h_k & h_{k-1} & \dots & h_0 \end{bmatrix}.$$

This matrix satisfies the orthogonality relation $GH^T = 0$.

Remark 9. Why reverse the coefficients? Since $g(X)h(X) = X^n - 1 \equiv 0 \pmod{X^n - 1}$ and every codeword has the form $c(X) = m(X)g(X)$, we have $c(X)h(X) \equiv 0$. Writing this condition as inner products of coefficient vectors leads precisely to the reversed-coefficient construction of H.

Example 17 (Parity-check matrix for a $[7, 4]_2$ cyclic code)**.** For $h(X) = 1+X+X^2+X^4$, the coefficient vector is $(1, 1, 1, 0, 1)$ and the reversed vector is $(1, 0, 1, 1, 1)$. Therefore

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

Example 18 (Verifying orthogonality)**.** Take the first row of G, $(1, 1, 0, 1, 0, 0, 0)$, and the first row of H, $(1, 0, 1, 1, 1, 0, 0)$. Their inner product is

$$1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 = 1 + 1 = 0 \pmod{2},$$

and similarly for all other row pairs, confirming $GH^T = 0$.

Example 19 (Error detection with the parity-check matrix)**.** For $\mathbf{v} = (1, 1, 1, 1, 1, 1, 1)$, we get $H\mathbf{v}^T = \mathbf{0}$, so no error is detected.

If instead $\mathbf{v}' = (0, 1, 1, 1, 1, 1, 1)$, then $H\mathbf{v}'^T \neq \mathbf{0}$, so an error is detected.

2.9 Systematic and Non-Systematic Encoding

Both methods produce valid codewords, but they place the original message differently inside the codeword.

Category	Non-systematic	Systematic
Encoding rule	$c(X) = m(X)g(X)$	$c(X) = p(X) + X^{n-k}m(X)$
Message position	Mixed throughout the codeword	Preserved explicitly in fixed positions
Ease of decoding	Requires division	Message bits read directly
Typical use	Rare	Common in real systems

Example 20 (Two encodings of the same message)**.** Let $g(X) = 1 + X + X^3$ and $\mathbf{m} = (1, 0, 1, 1)$.

Non-systematic: $c(X) = (1 + X^2 + X^3)(1 + X + X^3) = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6$, so $\mathbf{c} = (1, 1, 1, 1, 1, 1, 1)$; the message is not visibly preserved.

Systematic: the codeword has the form $(1, 0, 0, \mathbf{1}, \mathbf{0}, \mathbf{1}, \mathbf{1})$, so the last four bits are exactly the original message.

2.10 Practical Systematic Encoding

Systematic encoding proceeds in three steps.

Step 1. Shift the message polynomial: $X^{n-k}m(X)$.

Step 2. Compute the remainder: $p(X) = X^{n-k}m(X) \bmod g(X)$.

Step 3. Form the codeword: $c(X) = p(X) + X^{n-k}m(X)$.

Thus the codeword has the structure

$$[\underbrace{\text{parity}}_{n-k \text{ bits}} \mid \underbrace{\text{message}}_{k \text{ bits}}].$$

Remark 10. Since $p(X)$ is the remainder of $X^{n-k}m(X)$ modulo $g(X)$, the polynomial $X^{n-k}m(X) - p(X)$ is divisible by $g(X)$. Over $\mathbb{F}_2$, subtraction equals addition, so $c(X) = p(X) + X^{n-k}m(X)$ is also divisible by $g(X)$, hence a valid codeword.

Example 21 (Systematic encoding of $(1,0,1,1)$)**.** Let $g(X) = 1+X+X^3$, $n-k = 3$, $m(X) = 1+X^2+X^3$.

Step 1: $X^3m(X) = X^3 + X^5 + X^6$.

Step 2: Divide $X^6 + X^5 + X^3$ by $X^3 + X + 1$:

Step	Dividend	Quotient term
1	$X^6 + X^5 + X^3$	X^3
	$-(X^6 + X^4 + X^3) \;=\; X^5 + X^4$	
2	$X^5 + X^4$	X^2
	$-(X^5 + X^3 + X^2) \;=\; X^4 + X^3 + X^2$	
3	$X^4 + X^3 + X^2$	X
	$-(X^4 + X^2 + X) \;=\; X^3 + X$	
4	$X^3 + X$	1
	$-(X^3 + X + 1) \;=\; 1$	

So $p(X) = 1$.

Step 3: $c(X) = 1 + X^3 + X^5 + X^6$. Thus

$$\mathbf{c} = (\underbrace{1,0,0,}_{\text{parity}}\ \underbrace{1,0,1,1}_{\text{message}}).$$

Example 22 (Full table of systematic codewords for the $[7,4]_2$ code)**.**

Message ($m_0m_1m_2m_3$)	$m(X)$	Parity ($p_0p_1p_2$)	Codeword
0000	0	000	0000000
1000	1	110	1101000
0100	X	011	0110100
1100	$1+X$	101	1011100
0010	X^2	111	1110010
1010	$1+X^2$	001	0011010
0110	$X+X^2$	100	1000110
1110	$1+X+X^2$	010	0101110
0001	X^3	101	1010001
1001	$1+X^3$	011	0111001
0101	$X+X^3$	110	1100101
1101	$1+X+X^3$	000	0001101
0011	X^2+X^3	010	0100011
1011	$1+X^2+X^3$	100	1001011
0111	$X+X^2+X^3$	001	0010111
1111	$1+X+X^2+X^3$	111	1111111

2.11 Syndromes and Error Detection

Syndrome

Definition 9. The **remainder** obtained by dividing the received polynomial $v(X)$ by the generator polynomial $g(X)$ is called the **syndrome**:

$$s(X) = v(X) \bmod g(X).$$

- If $s(X) = 0$, no error is detected.
- If $s(X) \neq 0$, an error is detected.

Note. If $v(X) = c(X) + e(X)$ and $c(X)$ is a multiple of $g(X)$, then

$$s(X) = v(X) \bmod g(X) = e(X) \bmod g(X).$$

Thus the syndrome depends only on the error pattern, not on the original message.

Example 23 (Error-free reception)**.** Suppose $c(X) = 1 + X^3 + X^5 + X^6$ is received without error. Dividing by $X^3 + X + 1$ yields remainder 0, so the syndrome is 0.

Example 24 (Single-bit error at position X^2)**.** Suppose the received polynomial is $v(X) = 1 + X^2 + X^3 + X^5 + X^6$, so $e(X) = X^2$. Then $s(X) = X^2$, vector $(0, 0, 1)$. Since the syndrome is nonzero, the error is detected.

Example 25 (Syndrome table for single-bit errors in the $[7, 4]_2$ code)**.** For a single-bit error $e(X) = X^i$, the syndrome is $s(X) = X^i \bmod g(X)$.

Error position i	$e(X)$	$s(X) \bmod (1 + X + X^3)$	Syndrome vector
0	1	1	$(1, 0, 0)$
1	X	X	$(0, 1, 0)$
2	X^2	X^2	$(0, 0, 1)$
3	X^3	$1 + X$	$(1, 1, 0)$
4	X^4	$X + X^2$	$(0, 1, 1)$
5	X^5	$1 + X + X^2$	$(1, 1, 1)$
6	X^6	$1 + X^2$	$(1, 0, 1)$

Since all 7 syndromes are distinct, the code can correct **any single-bit error**.

Example 26 (Error correction using the syndrome)**.** Suppose the received vector is $\mathbf{v} = (1, 0, 1, 1, 0, 1, 1)$.

Step 1. With $v(X) = 1 + X^2 + X^3 + X^5 + X^6$, we obtain $s(X) = X^2$, syndrome vector $(0, 0, 1)$.

Step 2. From the syndrome table, $(0, 0, 1)$ corresponds to error position $i = 2$.

Step 3. $\hat{c}(X) = v(X) + X^2 = 1 + X^3 + X^5 + X^6$, so $\hat{\mathbf{c}} = (1, 0, 0, 1, 0, 1, 1)$.

Step 4. For a systematic code, the last four bits give $\mathbf{m} = (1, 0, 1, 1)$.

Example 27 (The case of a two-bit error)**.** Suppose $\mathbf{v} = (0, 1, 0, 1, 0, 1, 1)$, with errors at positions 0 and 1. Then $e(X) = 1 + X$ and $s(X) = 1 + X$, vector $(1, 1, 0)$. This matches the syndrome for a single error at position 3, so a single-error-correction decoder will miscorrect. This illustrates that the $[7, 4]_2$ code can correct 1 bit or detect 2 bits, but not correct arbitrary 2-bit errors.

2.12 Decoding of Cyclic Codes

The details of decoding depend on the specific family of cyclic code, but the overall structure is:

(1) **Syndrome computation.** Given $v(X) = c(X) + e(X)$, compute $s(X) = v(X) \bmod g(X)$ using an LFSR.

(2) **Error-pattern search.** Using $s(X)$, determine $e(X)$ via one of:

- *Syndrome lookup table*: for short codes, map syndrome directly to error pattern.
- *Meggitt decoder*: the syndrome register is shifted one clock at a time; hardware-friendly.
- *Algebraic decoding*: for BCH/RS codes, compute the error-locator polynomial (Berlekamp–Massey), locate roots (Chien search), and recover magnitudes (Forney algorithm).

(3) **Error correction.** Compute $\hat{c}(X) = v(X) - \hat{e}(X)$ (XOR for binary codes).

(4) **Message extraction.** For systematic codes, read the message positions; for non-systematic codes, divide $\hat{c}(X)$ by $g(X)$.

2.13 Error-Correcting Capability

A cyclic code is still a linear code, so its error-correcting capability is determined by its **minimum distance** $d_{\min}$.

Error-Correcting Capability

Theorem 8. *A code with minimum distance $d_{\min}$ can correct up to*

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

errors. If one is interested only in error ***detection****, then up to $d_{\min} - 1$ errors can be detected.*

Example 28 (Error-correcting capability of representative cyclic codes)**.**

Code	$[n, k]$	$d_{\min}$	Correction t	Detection
$[7, 4]_2$ cyclic (Hamming)	$[7, 4]_2$	3	1	2
$[15, 7]$ BCH code	$[15, 7]$	5	2	4
$[23, 12]$ Golay code	$[23, 12]$	7	3	6
$[15, 5]$ BCH code	$[15, 5]$	7	3	6
$[31, 21]$ BCH code	$[31, 21]$	5	2	4

Remark 11. For a general cyclic code there is usually no simple closed-form formula for $d_{\min}$ from $g(X)$. One must compute it by enumerating codewords or by deeper algebraic analysis.

BCH Bound

Theorem 9. *Let α be a primitive n-th root of unity in $\mathbb{F}_{q^m}$. If the generator polynomial $g(X)$ has the consecutive roots*

$$\alpha^b,\ \alpha^{b+1},\ \ldots,\ \alpha^{b+\delta-2},$$

then the minimum distance satisfies $d_{\min} \geq \delta$. The parameter δ is called the ***designed distance****.*

Remark 12. Because of the BCH bound, one can choose a target correction capability t in advance by setting $\delta = 2t + 1$. This is one of the main reasons BCH and Reed–Solomon codes are favored in practice over arbitrary cyclic codes.

Example 29 (Applying the BCH bound)**.** For the binary $[15, 7]$ BCH code, let α be a primitive 15th root of unity in $\mathbb{F}_{2^4}$. If $g(\alpha) = g(\alpha^2) = g(\alpha^3) = g(\alpha^4) = 0$, then there are 4 consecutive roots, so $\delta = 5$ and $d_{\min} \geq 5$, giving $t \geq 2$. This code attains $d_{\min} = 5$, so the BCH bound is tight.

2.14 Algebraic Structure of Cyclic Codes

Cyclic Codes Are Ideals

Theorem 10. *A cyclic code C of length n over $\mathbb{F}_q$ is a* ***principal ideal*** *of the quotient ring $R_n = \mathbb{F}_q[X]/\langle X^n - 1\rangle$. That is, $C = \langle g(X)\rangle$ for some polynomial $g(X)$.*

Proof. **Step 1:** C is an ideal.

- *Closed under addition:* by linearity, $a(X) + b(X) \in C$ whenever $a(X), b(X) \in C$.
- *Absorption:* cyclic-shift invariance gives $Xc(X) \in C$ for any $c(X) \in C$, hence $X^i c(X) \in C$ for all i, and by linearity $r(X)c(X) \in C$ for all $r(X) \in R_n$.

Step 2: The ideal is principal. Let $g(X)$ be the monic polynomial of smallest degree in C. For any $c(X) \in C$, write $c(X) = q(X)g(X) + r(X)$ with $\deg r < \deg g$. Since $r(X) = c(X) - q(X)g(X) \in C$ and $g(X)$ has minimal degree, $r(X) = 0$. Hence $C = \langle g(X)\rangle$. □

Example 30 (Checking the ideal structure for $[7,4]_2$ with $g(X) = 1 + X + X^3$)**.**

- $g(X) \in C$
- $Xg(X) = X + X^2 + X^4 \in C$
- $(1 + X)g(X) = 1 + X^2 + X^3 + X^4 \in C$

This illustrates both cyclic closure and absorption.

Example 31 (Why must $g(X)$ divide $X^n - 1$?)**.** For $g(X) = 1 + X + X^3$:

$$X^7 + 1 = (1 + X + X^3)(1 + X + X^2 + X^4) = g(X)h(X).$$

Thus $g(X) \mid X^7 + 1$, a necessary condition for any generator polynomial.

2.15 Classification of Cyclic Codes via Ideals

Cyclic codes correspond one-to-one with principal ideals of the quotient ring $R_n = \mathbb{F}_q[X]/\langle X^n - 1\rangle$.

Classification Theorem

Theorem 11. *Every ideal of R_n is a principal ideal generated by a monic divisor $g(X)$ of $X^n - 1$:*

$$I = \langle g(X)\rangle.$$

Hence there is a one-to-one correspondence:

$$g(X) \mid (X^n - 1) \quad \longleftrightarrow \quad \langle g(X)\rangle \text{ is a cyclic code of length } n \text{ over } \mathbb{F}_q.$$

Proof. The canonical projection $\pi : \mathbb{F}_q[X] \to R_n$ has kernel $\langle X^n - 1\rangle$. By the correspondence theorem for rings, ideals of R_n correspond bijectively to ideals J of $\mathbb{F}_q[X]$ satisfying $\langle X^n - 1\rangle \subseteq J$. Since $\mathbb{F}_q[X]$ is a PID, every such $J = \langle g(X)\rangle$ for a unique monic $g(X)$, and $\langle X^n - 1\rangle \subseteq \langle g(X)\rangle$ iff $g(X) \mid (X^n - 1)$. □

Remark 13 (Counting cyclic codes)**.** If $\gcd(n, q) = 1$ and $X^n - 1 = f_1(X)\cdots f_s(X)$ is a product of s distinct irreducible factors over $\mathbb{F}_q$, then every monic divisor corresponds to a subset $S \subseteq \{1, \dots, s\}$ via $g_S(X) = \prod_{i\in S} f_i(X)$. Therefore the **number of cyclic codes of length n over $\mathbb{F}_q$ is 2^s.**

Procedure for Determining Generator Polynomials

(1) Factor $X^n - 1$ completely into irreducible polynomials over $\mathbb{F}_q$.

(2) Form all products over subsets of those irreducible factors.

(3) Each generator polynomial $g(X)$ gives a cyclic code with parameters $[n,\ n - \deg g(X)]$.

Example 32 (All cyclic codes of length 7 over $\mathbb{F}_2$)**.** Since $X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3)$, there are $s = 3$ irreducible factors, giving $2^3 = 8$ cyclic codes:

Subset S	$g(X)$	$\deg g$	$[n, k]$	2^k	Remark
$\emptyset$	1	0	$[7, 7]$	128	Whole ring
$\{1\}$	$1 + X$	1	$[7, 6]$	64	Parity-check
$\{2\}$	$1 + X + X^3$	3	$[7, 4]$	16	Hamming
$\{3\}$	$1 + X^2 + X^3$	3	$[7, 4]$	16	
$\{1, 2\}$	$1 + X^2 + X^3 + X^4$	4	$[7, 3]$	8	
$\{1, 3\}$	$1 + X + X^2 + X^4$	4	$[7, 3]$	8	
$\{2, 3\}$	$1 + X + X^2 + X^3 + X^4 + X^5 + X^6$	6	$[7, 1]$	2	Repetition
$\{1, 2, 3\}$	$X^7 + 1$	7	$[7, 0]$	1	Zero ideal

The ideals form a lattice under inclusion: $g_1(X) \mid g_2(X)$ implies $\langle g_2(X)\rangle \subseteq \langle g_1(X)\rangle$.

Example 33 (Number of ideals for $n = 15$ over $\mathbb{F}_2$)**.** Since

$$X^{15} + 1 = (1 + X)(1 + X + X^2)(1 + X + X^4)(1 + X^3 + X^4)(1 + X + X^2 + X^3 + X^4),$$

there are $s = 5$ irreducible factors, so the number of cyclic codes is $2^5 = 32$.

Remark 14 (Caution when $\gcd(n, q) \neq 1$)**.** If $\gcd(n, q) \neq 1$, then $X^n - 1$ has repeated roots. For example, over $\mathbb{F}_2$,

$$X^6 + 1 = (1 + X)^2(1 + X + X^2)^2,$$

since $\gcd(6, 2) = 2$. The number of monic divisors becomes $\prod(e_i + 1) = (2 + 1)(2 + 1) = 9$, larger than $2^s = 4$.

2.16 Representative Families of Cyclic Codes

Code	Characteristic	Main applications
CRC	Error detection only; many standard polynomials	Ethernet, USB, ZIP, storage
BCH codes	Root-based construction; guaranteed designed distance	Flash memory, satellite comm.
Reed–Solomon (RS)	Nonbinary cyclic; symbol-level correction	CD/DVD/Blu-ray, QR codes, deep-space
Hamming codes (cyclic)	Cyclic form for certain parameters	ECC memory

Example 34 (CRC-8 generator polynomial)**.** A standard CRC-8 polynomial is $g(X) = X^8+X^2+X+1$.

- Parity length: 8 bits.
- Typical use: sensor networks and embedded systems.

Example 35 (CRC-32 used in Ethernet)**.**

$$g(X) = X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1.$$

32 parity bits protect the FCS (Frame Check Sequence) in Ethernet frames.

2.17 Quasi-Cyclic Codes and Post-Quantum Cryptography

Quasi-Cyclic Code

Definition 10. A linear code is called **quasi-cyclic** with index ℓ if it is closed under a cyclic shift by ℓ positions. If the code length is $n = m\ell$, then shifting any codeword by ℓ positions preserves membership. When $\ell = 1$, this reduces to an ordinary cyclic code.

Generator and parity-check matrices of quasi-cyclic codes are built from $m \times m$ **circulant blocks**. A circulant matrix is completely determined by its first row: each subsequent row is the previous row cyclically shifted right by one.

Example 36 (Structure of a circulant block)**.** The 4×4 circulant matrix with first row $(1, 0, 1, 1)$ is:

$$C = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}.$$

Storing only the first row suffices to reconstruct the entire matrix.

Use in HQC (Hamming Quasi-Cyclic)

HQC is a code-based key encapsulation mechanism designed to remain secure against quantum computers, including Shor's algorithm.

The classical problem: the McEliece cryptosystem is highly secure but has very large public keys (megabyte scale).

The HQC idea: adopt a quasi-cyclic structure to achieve:

(1) **Smaller public keys:** a circulant matrix is determined by one polynomial, reducing key size dramatically.

(2) **Faster arithmetic:** operations in $\mathbb{F}_2[X]/\langle X^n - 1\rangle$ admit efficient algorithms.

(3) **Provable security:** the scheme reduces to hard syndrome-decoding problems believed to be quantum-resistant.

A Exercises

References

[1] F. Oggier, *Teaching Material: Coding Theory*, feog.github.io. Available: `https://feog.github.io/`. Accessed: 2026-04-01.

[2] F. Oggier, *Coding Theory: Introduction*. Available: `https://feog.github.io/1-coding.pdf`. Accessed: 2026-04-01.

[3] F. Oggier, *Coding Theory: Linear Codes*. Available: `https://feog.github.io/2-coding.pdf`. Accessed: 2026-04-01.

[4] W. Cary Huffman and Vera Pless. *Fundamentals of error-correcting codes*. Cambridge University Press, 2003.